asgn07 - OOP Database Foundations: WNC Birds and Related Tables

Course: WEB-250
Points: 100
Application: WNC Bird Guide
Assignment focus: Create the database, connect through PDO, and display database information in an unstyled HTML table.

Overview

In the Bird Challenge, the data came from a file. We are now beginning the database section of the course. We will keep using Western North Carolina birds, but move the data into MySQL or MariaDB and use PHP Data Objects (PDO) to communicate with the database.

This assignment is deliberately smaller than a complete bird website. Build a **database-check page**, not a bird listing or image gallery. The page will identify the connected database and show how many records are in each of its three tables. Those counts must come from SQL, not from numbers typed into the HTML.

The supplied image files and their database records prepare us for later work. **Do not display the images, resize them, upload new ones, build bird detail pages, or add CSS.** Do not add searching, filtering, `array_filter()`, callbacks, or autoloading. An ordinary `foreach` loop is enough for the output in this assignment.

We are still using OOP: a `Database` class manages a PDO connection object. Bird model classes and row-to-object conversion will come after the connection and database structure are working.

References

Use these official references when a term or method is unfamiliar. You do not need to read every page from beginning to end.

Reference	Use it to look up
PHP: PDO MySQL DSN	The connection string and its settings.
PHP: PDO::query()	Running a fixed SQL statement.
PHP: PDOStatement::fetchColumn()	Retrieving a single value such as a count.
PHP: PDO error handling	Exceptions and connection/query errors.
PHP: htmlspecialchars()	Escaping values for HTML output.
MySQL: Foreign key constraints	Referential integrity and cascading actions.

What you should learn

By the end, you should be able to explain why a many-to-many relationship needs a linking table, connect to the database through a reusable static method, retrieve a value from a SQL result, and display it safely in HTML. You should also be able to distinguish an image file from the database record that describes it.

Part 1 - Understand the three-table design

The database is named `wnc_birds`. It contains two main tables and one junction table:

```
birds                bird_image_links                bird_images
-----                ----------------                -----------
id  <--------------  bird_id
                     image_id  -------------------->  id
species information   sex                            file_name
                     sort_order                      image metadata

One species -> many links <- many image files
One image file can also have links to more than one species.
```

`birds`: one record per species

Keep the original fields: `id`, `common_name`, `scientific_name`, `family_name`, `primary_habitat`, `seasonal_status`, and `description`.

The eight starter species are Eastern Towhee, American Robin, Blue Jay, Carolina Wren, Northern Cardinal, Red-bellied Woodpecker, Dark-eyed Junco, and Rose-breasted Grosbeak. A row describes a **species**, not one individually identified animal.

`bird_images`: one record per image file

This table holds an ID, a unique filename, a caption, alternate text, image type, credit, optional source URL, and pixel dimensions. It does **not** contain JPEG bytes, a BLOB, or a base64-encoded image.

For example, the filename `eastern_towhee_male.jpg` points to a file that is physically stored in `public/images/birds/`. Storing a filename does not upload the image or check that the file exists. We must keep the folder and the database records consistent.

`bird_image_links`: the many-to-many relationship

Each row contains a `bird_id` and an `image_id`. Both must refer to existing parent records. Together they form the primary key, so the same species cannot be linked to the same image twice.

A `bird_id` may appear in several rows, and an `image_id` may also appear in several rows. Neither column is individually unique. That is what allows **many-to-many**, rather than one-to-many.

The table also stores `sex` and `sort_order`. Sex belongs to the association because one image might contain several species or a male/female pair. The allowed labels are `male`, `female`, `both`, and `unknown`; use `unknown` rather than inventing certainty. In this starter set, labels describe the intended teaching illustrations.

A real shared-file example in the supplied data

`towhee_cardinal_comparison.jpg` contains side-by-side illustrations of an Eastern Towhee and a Northern Cardinal. There is **one image record** for that file, with **two linking records**: one for each species.

The two links do not require two copies of the image file. Combined with the separate male and female illustrations, the supplied data contains:

Table	Seed-only row count
`birds`	8
`bird_images`	17
`bird_image_links`	18

Without the junction table, putting a single `bird_id` inside `bird_images` would allow many images for one species, but not a single image shared by multiple species.

Referential integrity and cascades

Both foreign keys in `bird_image_links` use:

```
ON UPDATE CASCADE
ON DELETE CASCADE
```

The MySQL foreign-key reference above explains these actions. In this design:

Action on a parent record	What happens to the links	What is not deleted
Delete a species from `birds`.	Links for that species are removed.	Image records, other species, and the JPEG files.
Delete a record from `bird_images`.	Links to that image are removed.	Species records and the JPEG file.
Change a referenced parent ID.	The corresponding foreign-key values change.	Other parent records and files.

Deleting only a junction row removes only that relationship. The two parent records remain.

Cascading deletes are appropriate here because a relationship should not outlive a parent record. They do **not** run backward through the relationship and erase the other parent. The database also does not manage our filesystem: deleting an image record does not delete its JPEG. We will address file cleanup deliberately in a later assignment.

The auto-increment IDs should normally stay unchanged. Cascading updates are included, but renumbering primary keys is not a routine application task.

Part 2 - Set up the assignment folder

Unzip `asgn07_starter_files.zip`. Open the resulting `asgn07` folder in VS Code.

Typical local locations are:

```
macOS with MAMP:
/Applications/MAMP/htdocs/web250/asgn07

Windows with XAMPP:
C:\xampp\htdocs\web250\asgn07
```

A different established class workspace is fine. The folder names inside the project must match the following structure:

```
asgn07/
|-- asgn07.md
|-- README-START-HERE.md
|-- .gitignore
|-- private/
|   |-- .htaccess
|   |-- db_credentials.php.example
|   |-- functions.php
|   |-- initialize.php
|   `-- classes/
|       `-- Database.php
|-- public/
|   |-- index.php
|   `-- images/
|       `-- birds/
|           |-- eastern_towhee_male.jpg
|           |-- eastern_towhee_female.jpg
|           |-- ...remaining species images...
|           |-- towhee_cardinal_comparison.jpg
|           |-- IMAGE-NOTES.md
|           `-- image_manifest.json
`-- sql/
    |-- .htaccess
    |-- wnc_birds.sql
    `-- verify_setup.sql
```

You will complete `Database.php` and `index.php`, and create your local credentials file. The SQL, helper files, and images are supplied. Do not bring in the old `Bird` class, CSV parser, filter code, or `detail.php`.

About the image folder

There are 17 actual JPEG files, not just a list of links. These are **AI-generated teaching illustrations**, not wildlife photographs or a scientific identification guide. Read `IMAGE-NOTES.md` for their provenance and

limitations. The male/female labels describe the intended illustrations, not verified photographed individuals.

Leave the images in place and do not rename them. Their names match `bird_images.file_name` exactly. We have included dimensions for future resizing work, but **do not write image-display or image-processing code now**. You do not need to read the JSON manifest with PHP in this assignment.

Part 3 - Create and inspect the database

Start MySQL/MariaDB using your established local environment. Open phpMyAdmin, Adminer, or your usual database tool.

Read `sql/wnc_birds.sql`, then import it. This revised setup is **additive**: it does not drop tables or erase the earlier bird data. It creates missing tables and inserts missing starter records. Unique keys and no-op duplicate-key updates prevent the same seeds from being inserted again.

Back up existing work first. The setup is intended for a new database or the original, compatible one-table `wnc_birds` starter. It is not a universal migration script: `CREATE TABLE IF NOT EXISTS` does not change an incompatible table that already exists. Stop and ask for help if the database tool reports a schema or foreign-key error. Do not disable foreign-key checks to hide the problem.

Check the records before writing PHP

Select `wnc_birds` in the database tool and run:

```
SHOW TABLES;

SELECT COUNT(*) AS bird_count FROM birds;
SELECT COUNT(*) AS image_count FROM bird_images;
SELECT COUNT(*) AS link_count FROM bird_image_links;
```

For a fresh database, or the original database containing only the eight supplied species, expect **8, 17, and 18**. Existing extra records can make the totals larger. Do not delete valid earlier work just to match these numbers; mention the additional records in your submission notes.

Next run:

```
SHOW CREATE TABLE bird_image_links;
```

Locate the composite primary key, both foreign keys, and both cascading actions on each foreign key. The parent IDs and their foreign keys are all `INT UNSIGNED`. All three tables use InnoDB.

`sql/verify_setup.sql` supplies additional **read-only** checks, including a check for the shared image. Run it in the database tool. You are not being asked to build joins, a bird listing, or a filtering interface in PHP yet.

Checkpoint: Do not continue until all three tables exist, the sample data is present, and both foreign keys appear in the actual table definition.

Part 4 - Configure the local credentials

Copy `private/db_credentials.php.example` to `private/db_credentials.php`. Keep the example file unchanged. Update the new file using your local database settings:

```php
<?php
// Copy this file to db_credentials.php and use YOUR local server settings.
// Do not commit or submit the real credentials file.
const DB_HOST = '127.0.0.1';
const DB_PORT = 3306;
const DB_NAME = 'wnc_birds';
const DB_USER = 'your_database_user';
const DB_PASS = 'your_database_password';
```

`DB_PORT` is the **database port**, not the Apache or PHP web-server port. Use the port shown in your local database configuration. `127.0.0.1` specifies a TCP connection; the MySQL PDO DSN reference explains the distinction from a local socket connection.

The database already exists because you imported the SQL. PDO will connect to it; opening a connection is not the same as creating a database.

The provided `.gitignore` excludes `/private/db_credentials.php` from Git. It does **not** remove that file from an ordinary ZIP. You must leave it out of the submitted ZIP yourself. Never put real passwords in the example file, README, or screenshots.

For now, run locally. The database account used to import the schema needs creation privileges; the PHP page itself only performs read-only queries.

Part 5 - Complete the `Database` class

Open `private/classes/Database.php`. Keep the class, property, and method structure. Replace the TODO exception with the connection code so the completed file reads:

```php
<?php

class Database
{
    // The class holds one connection for this PHP request.
    // ?PDO allows either a PDO object or null (not connected yet).
    private static ?PDO $connection = null;

    public static function connect(): PDO
    {
        // Reuse the connection if this request has already created one.
```

```
        if (self::$connection instanceof PDO) {
            return self::$connection;
        }

        // The DSN identifies the database driver, server, port, and
  database.
        $dsn = 'mysql:host=' . DB_HOST
            . ';port=' . DB_PORT
            . ';dbname=' . DB_NAME
            . ';charset=utf8mb4';

        self::$connection = new PDO(
            $dsn,
            DB_USER,
            DB_PASS,
            [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]
        );

        return self::$connection;
    }
}
```

Read the important pieces

`private static ?PDO $connection = null;` means the property belongs to the class and is accessible only inside it. It starts as `null`; after connection it holds a PDO object. The `?` allows either value. We do not need to create a `Database` instance with `new Database()`.

`Database::connect()` is a static method call. The method checks for an existing connection before creating one. This shares one connection **within the current PHP request**; it is not a permanent connection shared across all visitors.

The **DSN**, or Data Source Name, identifies the driver, server, port, database, and character set. The username and password are supplied as separate arguments to PDO. For example, a DSN could be:

```
mysql:host=127.0.0.1;port=3306;dbname=wnc_birds;charset=utf8mb4
```

The three options make errors become exceptions, establish associative arrays as the default row-fetch format, and disable PDO's emulated prepared statements. We will use prepared statements when queries receive changing values in a later assignment. The fixed queries in this assignment contain no visitor-supplied values.

`private/initialize.php` already loads the credentials, helper function, and class with `require_once`. Read its comments. It deliberately does not open the connection, so the page can do that inside a `try` block. There is no autoloading callback.

Part 6 - Test the connection before building the table

Temporarily replace the contents of `public/index.php` with:

```php
<?php
require_once __DIR__ . '/../private/initialize.php';

try {
    $database = Database::connect();
    $statement = $database->query('SELECT DATABASE()');
    $database_name = (string) $statement->fetchColumn();

    echo '<p>PDO connection successful.</p>';
    echo '<p>Database: ' . h($database_name) . '</p>';
} catch (PDOException $exception) {
    http_response_code(500);
    error_log('asgn07 database error: ' . $exception->getMessage());
    echo '<p>Connection failed. Check the local PHP error log.</p>';
}
```

Use your established PHP/Apache setup, or start PHP's local development server from the **asgn07 project root** in the VS Code terminal:

```
php -S localhost:8000 -t public
```

Open:

```
http://localhost:8000/
```

Keep that terminal running. `-t public` makes only `public/` the document root. The database server must still be running; PHP's web server does not start MySQL. Stop the development server with **Ctrl+C** when finished. PHP documents this server as a development tool, not a production server.

When using local Apache instead, open the URL for this project's `public/index.php` using your configured web-server port. The supplied `.htaccess` files are only an additional Apache safeguard where overrides are supported; they are not a replacement for serving only `public/` on a production host.

Expected text:

```
PDO connection successful.
Database: wnc_birds
```

A generic message appears in the browser on failure. The detailed exception goes to the PHP error log, which appears in the terminal when using the built-in development server. Do not echo raw database exceptions to visitors.

Checkpoint: Fix connection errors now. Do not add the table until this test works.

Part 7 - Display a database summary in an HTML table

First, understand this sequence:

```
$statement = $database->query('SELECT COUNT(*) FROM birds');
$bird_count = (int) $statement->fetchColumn();
```

`COUNT(*)` asks the database to count its rows. `query()` runs the fixed statement and returns a `PDOStatement` object. `fetchColumn()` retrieves one column from the next result row; here that one value is the count. Casting to `(int)` makes the PHP count explicitly an integer. A count of zero is a legitimate value, not a failed query.

The following is the same operation written as one expression:

```
$bird_count = (int) $database->query('SELECT COUNT(*) FROM birds')-
>fetchColumn();
```

Do not use `PDOStatement::rowCount()` to determine how many rows a SELECT represents. We are explicitly asking SQL for the count.

Replace the temporary connection page

Replace **all** the temporary code in `public/index.php` with the following program. Do not append a second page beneath it.

```
<?php
require_once __DIR__ . '/../private/initialize.php';

$database_name = '';
$summary = [];
$error_message = null;

try {
    $database = Database::connect();

    // Each query returns one value. fetchColumn() retrieves that value.
    $database_name = (string) $database->query('SELECT DATABASE()')-
>fetchColumn();
    $bird_count = (int) $database->query('SELECT COUNT(*) FROM birds')-
>fetchColumn();
```

```
    $image_count = (int) $database->query('SELECT COUNT(*) FROM
bird_images')->fetchColumn();
    $link_count = (int) $database->query('SELECT COUNT(*) FROM
bird_image_links')->fetchColumn();

    // The labels are fixed; every numeric count comes from the database.
    $summary = [
        ['table_name' => 'birds', 'row_count' => $bird_count, 'purpose' =>
'Species records'],
        ['table_name' => 'bird_images', 'row_count' => $image_count,
'purpose' => 'Image file metadata'],
        ['table_name' => 'bird_image_links', 'row_count' => $link_count,
'purpose' => 'Bird-to-image relationships'],
    ];
} catch (PDOException $exception) {
    http_response_code(500);
    error_log('asgn07 database error: ' . $exception->getMessage());
    $error_message = 'Database check failed. Check the server, credentials,
and imported tables. See the local PHP error log for details.';
}
?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>asgn07 - WNC Birds Database Check</title>
</head>
<body>
    <h1>WNC Birds Database Check</h1>

    <?php if ($error_message !== null): ?>
        <p><?= h($error_message) ?></p>
    <?php else: ?>
        <p>PDO connection and database queries: successful.</p>
        <p>Database: <?= h($database_name) ?></p>

        <table>
            <caption>Database record counts</caption>
            <thead>
                <tr>
                    <th scope="col">Table</th>
                    <th scope="col">Rows</th>
                    <th scope="col">Purpose</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($summary as $item): ?>
                    <tr>
                        <th scope="row"><?= h($item['table_name']) ?></th>
                        <td><?= h((string) $item['row_count']) ?></td>
                        <td><?= h($item['purpose']) ?></td>
                    </tr>
                <?php endforeach; ?>
```

```
            </tbody>
        </table>

        <p>The image files are saved for a future assignment. No bird list,
image gallery, or resizing is included here.</p>
    <?php endif; ?>
</body>
</html>
```

What the loop is doing

`$summary` is a small array of three display records. The table names and purpose labels are fixed text; the numerical values came from the database. `foreach` visits each summary record and produces one HTML row. It is an ordinary loop, not a callback.

The `h()` helper wraps `htmlspecialchars()` to escape values for HTML. Its argument is a string, so the count is converted with `(string)` when displayed. SQL query safety and HTML output escaping solve different problems.

The `<caption>`, `<thead>`, `<tbody>`, and scoped header cells give the table meaningful structure. **Do not add a stylesheet, a `<style>` element, inline styles, or a CSS framework.** An unstyled table may have no visible cell borders; that is expected.

Expected browser result

The final page should show the database name and a table with these values for the supplied seeds:

Table	Rows	Purpose
birds	8	Species records
bird_images	17	Image file metadata
bird_image_links	18	Bird-to-image relationships

The page does not show individual birds, image thumbnails, a filename gallery, or detail links. The image metadata is present in the database for future assignments; this page only reports its record count. The count also does not prove that the physical files exist, which is why the folder inspection remains a separate check.

Part 8 - Test two important behaviors

Re-import without duplicating the seeds

Run `sql/wnc_birds.sql` again and refresh the page. The original supplied records should not duplicate. With seed-only data, the counts remain 8, 17, and 18. Existing edits to seeded records are preserved; re-import is not a reset button.

Observe a connection failure

Temporarily change `DB_NAME` to a database name that does not exist. Reload the page and confirm that the generic failure message appears, rather than a successful connection message or a misleading table of zeroes. Read the detailed error in the local PHP error log.

Restore `DB_NAME` to `wnc_birds` and verify success again. Submit the working version, not the broken experiment.

Do not test deletion by removing real course data. The supplied verification SQL can inspect the foreign-key rules without modifying records.

Part 9 - Explain the design

Create `README.md` in the project root. Answer each question in two or three sentences:

1. Why does this many-to-many design use three tables instead of putting one `bird_id` in `bird_images`?
2. What happens to the links, image records, and physical JPEG files when a species record is deleted?
3. What does the PDO DSN contain, and why is `Database::$connection` static?
4. What is the difference between `query()` and `fetchColumn()` in the count example?

Also record the three counts from the working page. Mention any legitimate pre-existing records that make your counts different from the supplied seed-only totals. Explain briefly what happened during the incorrect-database-name test. Do not include a password or raw error log containing local account details.

Troubleshooting

Symptom	Check
`could not find driver` in the error log	The PHP installation serving the page needs PDO's MySQL driver. In the same terminal environment, run `php -r "print_r(PDO::getAvailableDrivers());"` and look for `mysql`. CLI and Apache may use different PHP configurations.
Connection refused	Confirm that MySQL/MariaDB is running and that `DB_HOST` and `DB_PORT` describe that server, not the web server.
Access denied	Check the database username, password, and permissions.
Unknown database	Import the SQL and check `DB_NAME`. Restore it after the failure experiment.
Missing table	Confirm that the full updated SQL was imported into the database used by PDO.
Foreign-key creation error	Compare the actual parent/child column types and InnoDB engines. A pre-existing incompatible table is not repaired by `IF NOT EXISTS`. Do not turn off integrity checks.

Symptom	Check
Counts larger than expected	Inspect the existing data. The setup preserves earlier records; it does not wipe the database.
Images do not appear	Correct for this assignment. Check for the files in the folder, not an image gallery in the browser.
Plain table with no styling	Correct. CSS is intentionally deferred.

The PDO driver list and PHP development-server documentation explain the corresponding checks above.

Submission

Submit a ZIP of the completed assignment folder named:

```
asgn07_yourLastName.zip
```

Include the completed PHP files, `README.md`, both SQL files, the original example credentials file, and the supplied image folder with its notes and manifest. **Exclude `private/db_credentials.php`.** Make a separate submission copy if needed so the local working version keeps its credentials. No screenshots or live webhost deployment are required for this assignment.

Before submission, verify that the page is working again after the error experiment, that all 17 JPEG files are present, and that no real password appears in the archive.

Grading rubric - 100 points

Category	Points	What is evaluated
Database and seed data	20	Three correct tables; original species data retained; image metadata and relationship rows present.
Referential integrity	15	Both foreign keys use cascading updates/deletes; composite primary key prevents duplicate links; shared-file relationship is present.
PDO and OOP connection	25	Correct DSN, options, static connection storage, connection reuse, and credentials kept separate.
SQL retrieval and HTML output	20	All three counts come from SQL; database name is retrieved; an ordinary loop produces a semantic HTML table with escaped output.
Explanation and testing	10	README explains the relationship and PDO calls; repeat import and connection-error experiment are completed and restored.

Category	Points	What is evaluated
Organization and submission	10	Required files and all images included; no real credentials; no CSS, filters, callbacks, bird listing, or image gallery added.
Total	**100**	

Looking ahead

The groundwork is now present: species records, separate image metadata, links between them, and image files on disk. Later assignments can retrieve individual records, introduce model classes and prepared statements, display associated images, and create resized copies. Keep those extensions out of asgn07 so the database connection and relationships remain the focus.